

Lock-Free алгоритмы с Interlocked и Volatile

C# Developer. Advanced



Проверить, идет ли запись

Меня хорошо видно & слышно?





Антон Орехов

- >20 лет опыта разработчиком (C#, Python, SQL)
- программист интеграции

 @Antoni_www

 <mailto:Anton@yandex.ru>

Карта курса



СТАРТ

Управление памятью
на уровне ядра

Асинхронность и
многопоточность
для экспертов

Внутренние
механизмы CLR

Профилирование и
экстремальная
оптимизация

Современные
парадигмы и
инструменты эксперта

Проектный
модуль

ФИНИШ



Цели вебинара

К концу занятия вы сможете

1. Научиться разрабатывать потокобезопасные структуры данных без использования блокировок;
2. Применять класс Interlocked для атомарных операций над примитивными типами;
3. Понимать роль Volatile и барьеров памяти для обеспечения корректности многопоточного кода.



Цели вебинара

К концу занятия вы сможете

1. Разрабатывать потокобезопасные структуры данных без блокировок

2. Применять класс Interlocked

3. Понимать роль Volatile

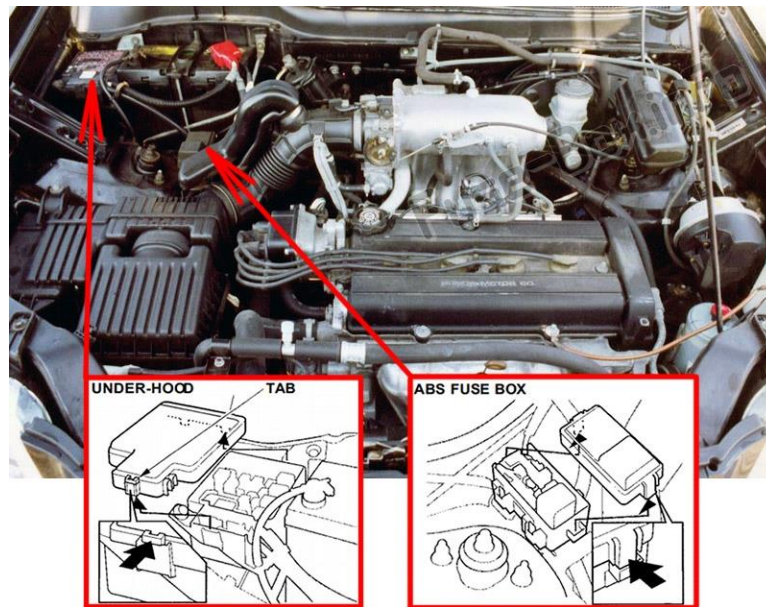
Смысл

Для чего вам это уметь

Понимать низкоуровневые механизмы синхронизации и принимать обоснованные решения при проектировании высокопроизводительных систем



Look «under the hood»



Маршрут вебинара

Введение в lock-free программирование

Атомарные операции (класс Interlocked)

Барьеры памяти и модель памяти .NET

Ключевое слово volatile и класс Volatile

Сравнение производительности

Рефлексия



Тестовый пример

Тестовый пример можно скачать по ссылке:

<https://github.com/Dzitskiy/LockFreeAlgorithms.git>



Введение в lock-free программирование: зачем нужно, преимущества и недостатки

Что не так с lock-based подходами?

Несколько проблем, которые в определенных сценариях могут быть критичными:

- Взаимная блокировка (deadlock);
- Динамическая взаимоблокировка (livelock);
- Голодание (starvation);
- Инверсия приоритетов (priority inversion);
- Производительность при высокой конкуренции (contention);
- Риск неудачного дизайна блокировок:
 - Слишком крупные блокировки (coarse-grained);
 - Слишком мелкие (fine-grained).

Проблемы, связанные с блокировками (такие как deadlock, livelock, race condition) часто сложно воспроизвести и отладить.



Классический пример

```
// Классический deadlock
object lock1 = new object();
object lock2 = new object();

// Поток 1
lock (lock1)
{
    Thread.Sleep(100);
    lock (lock2) // Ждет lock2, но его держит Поток 2
    {
        // ...
    }
}

// Поток 2
lock (lock2)
{
    Thread.Sleep(100);
    lock (lock1) // Ждет lock1, но его держит Поток 1
    {
        // ...
    }
}
```

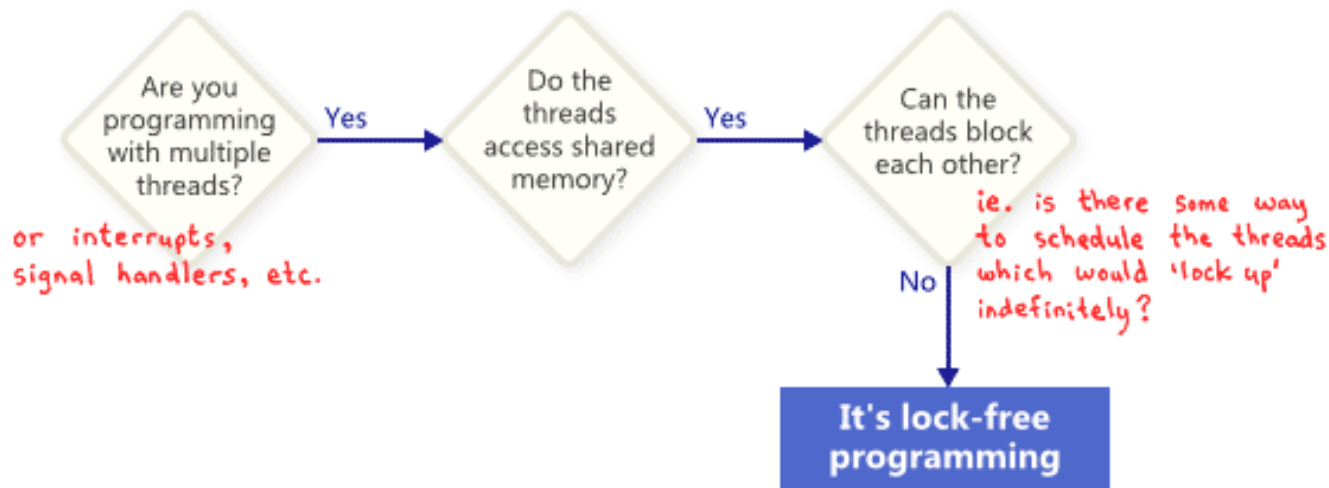
Lock-free алгоритмы

Lock-free алгоритмы в C# — это методы синхронизации, которые избегают использования блокировок (в частности, мьютексов) для защиты общих ресурсов, достигая максимальной производительности и линейной масштабируемости на многопроцессорных системах.

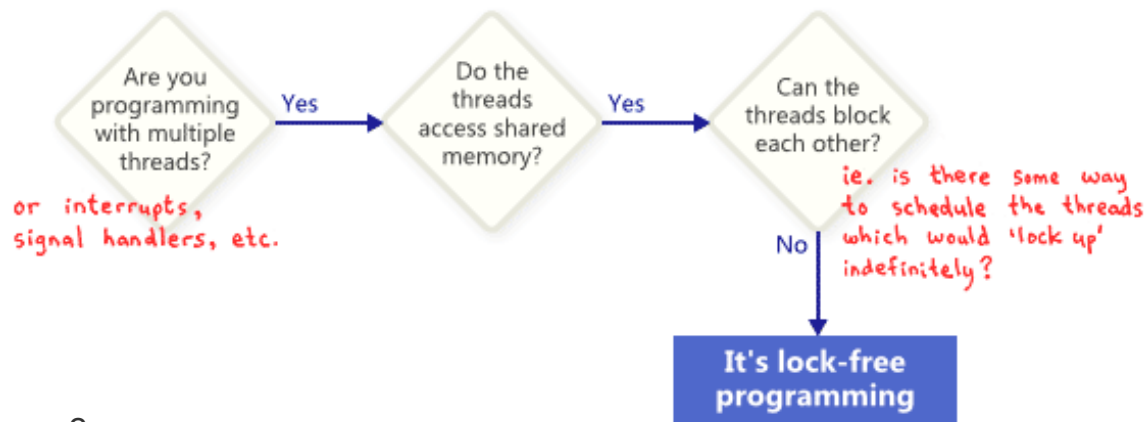
Вместо блокировок они используют атомарные операции, такие как `Interlocked` в .NET, позволяя потокам работать с данными без блокировки доступа других потоков.



Lock-free алгоритмы

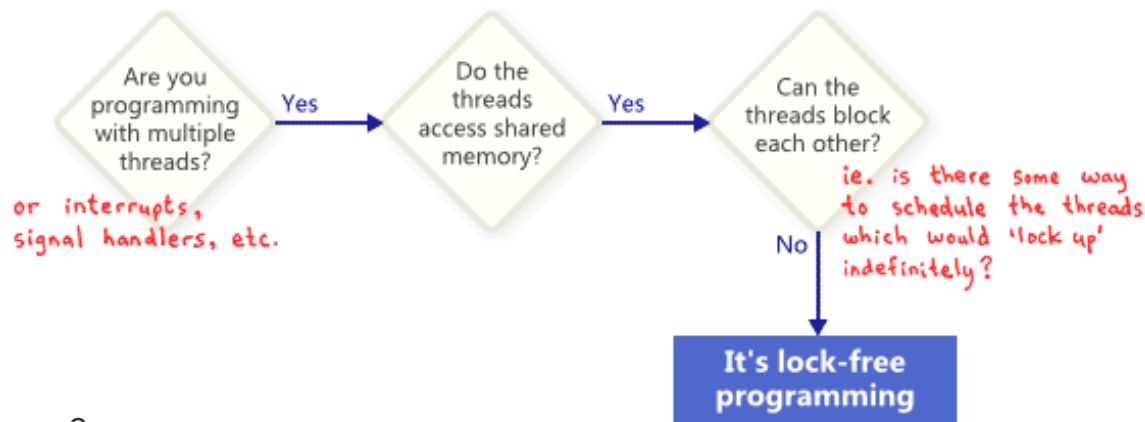


Lock-free алгоритмы



Нужно ответить всего на 3 вопроса...

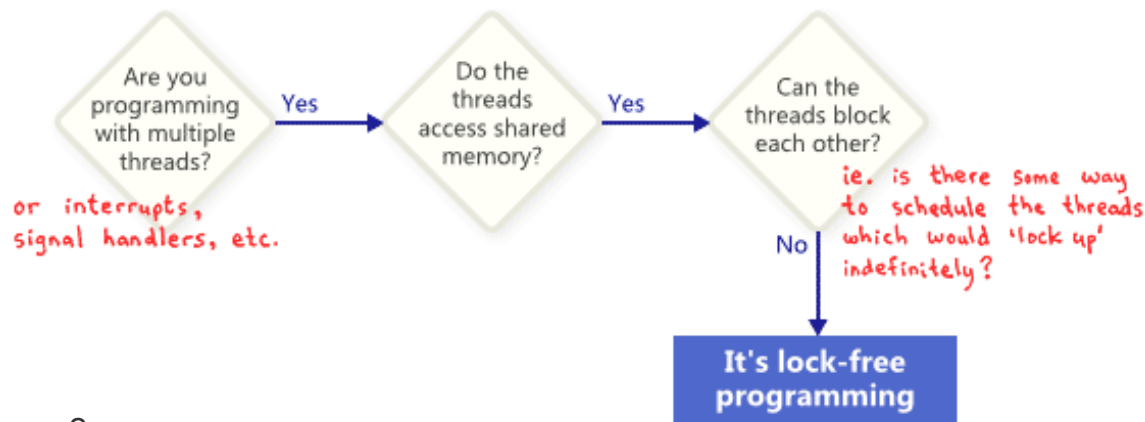
Lock-free алгоритмы



Нужно ответить всего на 3 вопроса:

- Мы работаем с несколькими потоками?

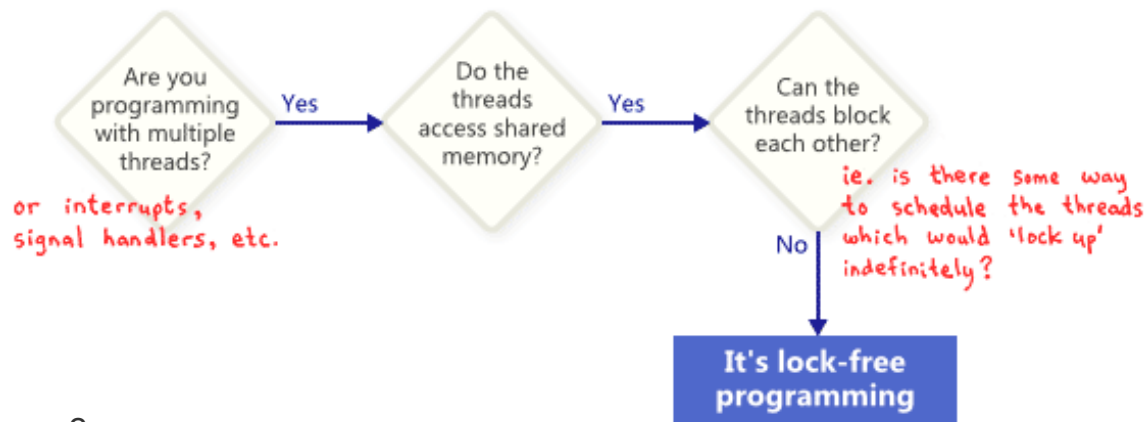
Lock-free алгоритмы



Нужно ответить всего на 3 вопроса:

- Мы работаем с несколькими потоками?
- Потоки имеют доступ к разделяемой памяти?

Lock-free алгоритмы



Нужно ответить всего на 3 вопроса:

- Мы работаем с несколькими потоками?
- Потоки имеют доступ к разделяемой памяти?
- Могут ли потоки блокировать друг друга?

Lock-free алгоритмы

Lock-free алгоритмы в C# — это методы синхронизации, которые избегают использования блокировок (в частности, мьютексов) для защиты общих ресурсов, достигая максимальной производительности и линейной масштабируемости на многопроцессорных системах.

Вместо блокировок они используют атомарные операции, такие как `Interlocked` в .NET, позволяя потокам работать с данными без блокировки доступа других потоков.



Lock-free алгоритмы

Lock-free алгоритмы в C# — это методы синхронизации, которые избегают использования блокировок (в частности, мьютексов) для защиты общих ресурсов, достигая максимальной производительности и линейной масштабируемости на многопроцессорных системах.

Вместо блокировок они используют **атомарные операции**, позволяя потокам работать с данными без блокировки доступа других потоков.



Что такое атомарные операции?

Атомарная операция — это операция, которая выполняется как единое целое, без возможности прерывания ее выполнения другими потоками.

В процессе выполнения такой операции состояние системы не может быть изменено другим потоком, что гарантирует, что операция либо завершится полностью, либо не начнется вовсе.

В lock-free алгоритмах атомарные операции используются для обеспечения безопасности при работе с общими данными без использования блокировок.



Чем обусловлена атомарность?

Атомарность обеспечивается на уровне аппаратного обеспечения (процессора) или на уровне компилятора/языка программирования. Такие операции компилируются в специальные инструкции процессора, которые гарантируют, что операция будет выполнена как одна неделимая инструкция (выполняются за один такт и не могут быть прерваны).

В современных процессорах многие операции уже атомарны (например, операции чтения/записи простых типов).

В C# для работы с атомарными операциями предоставляется класс `Interlocked`, который содержит статические методы для атомарных операций над различными типами данных.



Lock-free алгоритмы

На самом деле всё немного сложнее...

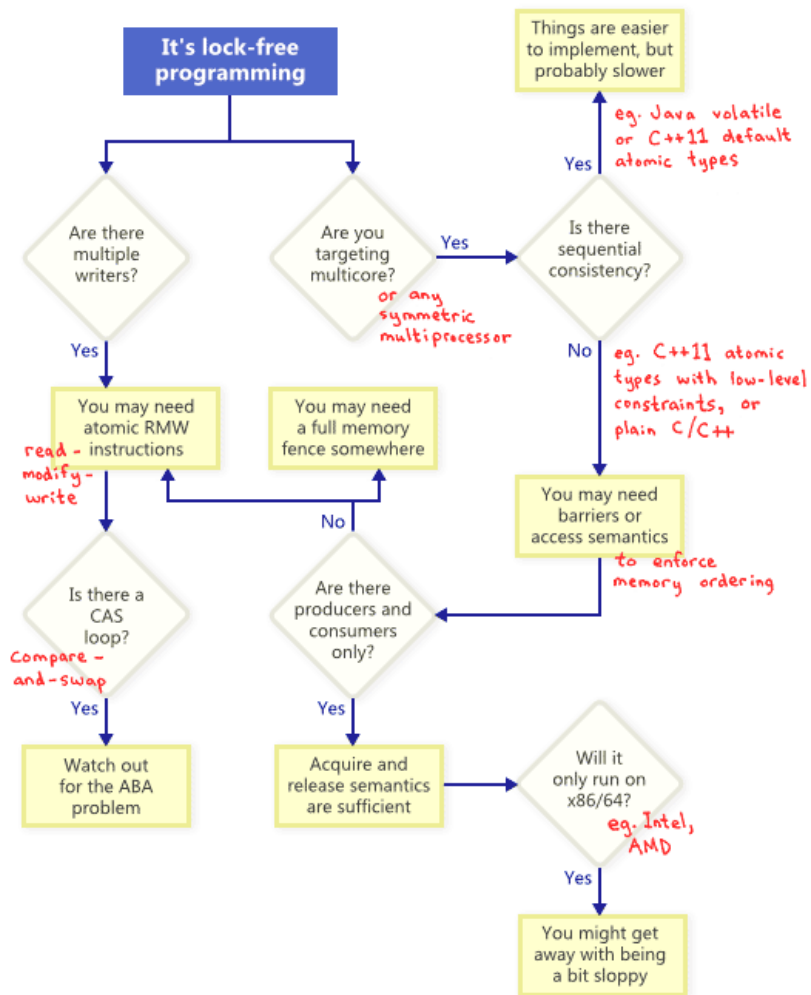


Lock-free алгоритмы

На самом деле всё немного сложнее...

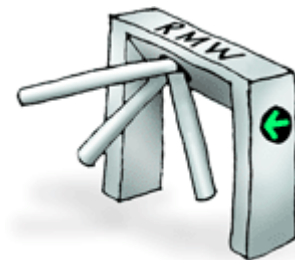
Разберемся в понятиях:

- RWM
- CAS
- ABA problem



Операции атомарного изменения (RMW)

RMW (Read-Modify-Write) — это атомарные операции, которые читают значение из памяти, изменяют его и записывают обратно в одну неделимую операцию.



Все три шага выполняются атомарно — без возможности вмешательства других потоков.

Преимущества RMW-операций:

- Исключение гонок данных
- Высокая производительность
- Отсутствие deadlock'ов

Compare-And-Swap (CAS)

Compare-And-Swap (CAS) — это атомарная инструкция, используемая в многопоточном программировании для реализации синхронизации без блокировок.

CAS является частным случаем RMW-операций.

CAS выполняет следующие шаги атомарно:

- Сравнивает значение в памяти с ожидаемым значением
- Если значения совпадают, то записывает новое значение в память
- Если значения не совпадают, то ничего не делает



Compare-And-Swap (CAS)

```
// Логика CAS (псевдокод)
public static bool CompareAndSwap(ref int location, int expected, int newValue)
{
    if (location == expected) // Сравниваем
    {
        location = newValue; // Обмениваем
        return true;
    }
    return false;
}
```



CAS - самая важная RMW операция

- CAS - это RMW операция с условием
- Типичный паттерн использования - CAS-цикл
- Все простые RMW операции могут быть реализованы через CAS-циклы
- Практически все lock-free структуры данных строятся на CAS

CAS - универсальный строительный блок для всего параллельного программирования без блокировок.

Для большинства задач лучше использовать высокоуровневые конструкции из `System.Collections.Concurrent`, но понимание CAS необходимо для создания собственных высокопроизводительных lock-free алгоритмов.



ABA-проблема

ABA-проблема:

Если значение изменяется туда и обратно между проверкой и записью, операция сравнения по ссылке считает состояние неизменным.

Это одна из самых сложных проблем для lock-free алгоритмов, поэтому при самостоятельной разработке всегда применяют добавление дополнительного счётчика/тега (version/tag), инкрементируемого при каждом изменении значения, расширенные атомарные проверки, либо готовые примитивы синхронизации из стандартных библиотек.



Основные способы решения АВА-проблемы

- Дополнительный тэг или версионный счётчик
- Double-width CAS (Compare-And-Swap)
- Hazard pointers
- Epoch-based reclamation
- Использование специализированных коллекций .NET

Множество конкурентных структур данных из `System.Collections.Concurrent` уже реализуют вышеописанные методы защиты от АВА-проблемы во внутренней логике.



Промежуточные итоги

- Lock-free алгоритмы позволяют избегать блокировок, что может повысить производительность и избежать проблем deadlock, livelock и т.д.
- Однако они сложнее в реализации и отладке.
- Используются в высокопроизводительных сценариях, когда борьба за блокировки становится узким местом.

How It Works

Lock-free алгоритмы позволяют обойти проблемы Lock-based, связанным с блокировками, благодаря использованию **атомарных операций** для синхронизации доступа к общим данным. Вместо того чтобы захватывать блокировки, потоки пробуют выполнить атомарные **RMW**-операции, в частности - операцию сравнения и замены (**CAS**), повторяя попытку в цикле до успешного выполнения. Такой подход гарантирует, что хотя бы один поток сможет продвигаться вперёд, предотвращая взаимные блокировки и бесконечные ожидания (при этом нужно помнить о дополнительных проверках от **ABA-проблемы**).



Вопросы для проверки

1. В чем суть **lock-free** алгоритмов?
2. Что такое **atomic operations**?
3. Что такое **RMW**?
4. Почему особую важность имеет **CAS**?
5. Что такое **ABA problem**?



How It Works

Неблокирующие алгоритмы синхронизации

concurrencyfreaks.blogspot.com
Pedro Ramalhete



Плюсы:

- Нет условий для возникновения состояние взаимной блокировки.
- Нет корреляции между потоками.

Минусы:

- Сложны в понимании.
- Сложны в анализе и реализации.
- Сложны в тестировании на корректность.

**Атомарные операции:
класс Interlocked.**

System.Threading.Interlocked



System.Threading.Interlocked

System.Threading.Interlocked — это класс .NET, предоставляющий набор методов для атомарных (неделимых) операций над переменными, которые могут быть разделены между потоками.

Характеристики и применение

- Методы Interlocked работают с простыми типами (int, long, float, object references) и гарантируют корректность в многопоточной среде.
- Все операции выполняются быстро и без блокировок, что идеально для lock-free программирования и конкурентных алгоритмов.
- Не синхронизируются с обычными операциями присваивания или арифметическими операциями, поэтому всегда следует использовать только атомарные методы при доступе к переменным между потоками.

Класс Interlocked необходим для обеспечения безопасности данных и высокой производительности в многопоточных C#-приложениях, где требуется минимизация задержек и блокировок

Атомарные операции на C#

В C# для lock-free алгоритмов используются атомарные операции, предоставляемые через класс ***System.Threading.Interlocked***.

Ключевой операцией является CAS (Compare-And-Swap), которая позволяет безопасно проверять и изменять значение переменной без блокировок.

Основные атомарные операции для lock-free:

- *CompareExchange (CAS)*
- *Exchange*
- *Increment/Decrement*
- *Add*
- *Read/Write*



Какие типы данных в C# можно использовать с атомарными операциями

В C# с атомарными операциями, реализуемыми через класс `Interlocked`, можно безопасно использовать следующие типы данных:

- **int, uint, long, ulong** — атомарные операции гарантированы через методы `Interlocked`
- **float** — атомарная запись и чтение поддерживаются, но модификации требуют осторожности
- **bool, char, byte, sbyte, short, ushort** — чтение и запись атомарны из-за их размера
- **Ссылочные типы** (`object`, указатели на объекты)
- **Enum** — если базовый тип `enum` — один из вышеперечисленных.

Для типов **double** и **long** атомарные операции гарантированы только при правильном выравнивании данных и зависят от архитектуры системы.

Для **структур** и **пользовательских типов** атомарные операции не поддерживаются напрямую: требуется использовать атомарные операции над отдельными полями поддерживаемых типов или реализовывать специальные lock-free механизмы.



LIVE

Важные ограничения

- Методы Interlocked работают атомарно только с одной переменной. Для сложных структур или нескольких переменных потребуются другие механизмы.
- Interlocked операции быстрее блокировок при низкой конкуренции, но создают барьер памяти. При высоких конфликтах спин-локи на основе Interlocked могут значительно нагружать CPU.
- Interlocked идеален для счетчиков, флагов и простых обновлений. Для сложных многошаговых операций над несколькими переменными часто лучше подходит lock.
- Прямая поддержка есть для int, long и ссылочных типов. Для double и float требуется обходной путь с CompareExchange и циклом повтора.

Класс Interlocked

- Атомарные операции: Increment, Decrement, Add, Exchange, CompareExchange
- CompareExchange можно использовать как «строительный блок» для сложных операций
- Ограничения: работа только с примитивами

Вопросы для проверки

1. Когда оправдано применение класса `Interlocked`?
2. Какие есть ограничения?
3. Что такое операций `atomic types`?



Вопросы



если есть вопросы



если вопросов нет

Модель памяти и барьеры в .NET

Модель памяти в .NET

Модель памяти .NET - это набор правил, определяющих как потоки видят изменения в памяти, сделанные другими потоками. Это слабая модель памяти (weak memory model), которая допускает переупорядочивание операций для повышения производительности.

Цель модели памяти - обеспечить взаимодействие потоков. Когда один из потоков записывает значения в память, а другой - считывает из памяти, модель памяти диктует, какие значения может увидеть читающий поток.

Проблема: Без специальных указаний компилятор и процессор могут:

- Переупорядочивать инструкции для оптимизации
- Кэшировать значения в регистрах процессора
- Откладывать запись в основную память

Барьеры памяти (Memory Barriers)

Барьеры памяти - это специальные инструкции, которые предотвращают переупорядочивание операций чтения/записи памяти компилятором и процессором.

Типы барьеров в .NET:

```
// Thread.MemoryBarrier()
// Полный барьер - предотвращает переупорядочивание операций через барьер
Thread.MemoryBarrier();

// Interlocked операции
// Создают полные барьеры памяти
Interlocked.Increment(ref value);

// Volatile класс
Volatile.Read(ref value);
Volatile.Write(ref value);

// Атрибут [Volatile]
[Volatile] private int volatileField;
```



Частичные и полные барьеры памяти в .NET

- **Полные барьеры памяти (Full Memory Barrier)**

Полный барьер (также называемый "full fence") - самый сильный тип барьера, он гарантирует:

- Все операции записи до барьера завершаются и становятся видимыми
- Все операции чтения после барьера видят актуальные значения
- Никакие операции не могут быть переупорядочены через барьер

- **Частичные барьеры памяти (Partial Memory Barriers)**

Частичные барьеры предоставляют более слабые гарантии и быстрее выполняются:

1. Барьер записи (**Write Barrier**)

- Гарантирует, что все записи до барьера завершатся до записей после барьера
- Не гарантирует видимость для других потоков

2. Барьер чтения (**Read Barrier**)

- Гарантирует, что все чтения после барьера увидят актуальные значения
- Не предотвращает переупорядочивание записей

Ключевые различия

Характеристика	Полный барьер	Частичный барьер
Гарантии	Полная упорядоченность	Частичная упорядоченность
Производительность	Медленнее	Быстрее
Использование	Сложная синхронизация	Простые флаги и состояния
Примеры	lock, Interlocked	volatile, Volatile.Read/Write

Виды барьеров памяти:

- Полный барьер (full fence): Thread.MemoryBarrier(), Interlocked операции
- Барьер чтения: Volatile.Read(), volatile поля
- Барьер записи: Volatile.Write(), volatile поля



Связь Interlocked операций с барьерами

Все операции Interlocked создают полные барьеры памяти (full fence).

Это означает, что:

- Никакие операции не могут быть перемещены до барьера
- Никакие операции не могут быть перемещены после барьера
- Все изменения памяти становятся видимыми всем процессорам



Сводная таблица

Операция	Атомарность	Барьер памяти	Производительность
lock	Да	Полный барьер	Низкая
Interlocked	Да	Полный барьер	Средняя
Volatile.Read/Write	Нет	Частичный барьер	Высокая
Thread.MemoryBarrier	Нет	Полный барьер	Высокая
Обычная операция	Нет	Нет барьера	Максимальная

LIVE

Ключевые принципы

Когда использовать:

- Interlocked - для атомарных операций
- Volatile - для флагов и простых состояний
- Thread.MemoryBarrier() - для сложных сценариев синхронизации

Правила видимости:

- Операции до барьера завершаются до операций после барьера
- Изменения становятся видимыми всем процессорам после барьера
- Барьеры предотвращают переупорядочивание операций

Производительность:

- Барьеры памяти имеют стоимость производительности
- Используйте минимально необходимый уровень синхронизации
- Interlocked операции быстрее lock'ов но медленнее обычных операций

Промежуточные итоги

Барьеры памяти — это мощный инструмент для обеспечения видимости изменений и порядка операций в многопоточных программах. Однако их следует использовать с осторожностью, так как неправильное использование может привести к трудноотлавливаемым ошибкам.

В большинстве случаев лучше использовать высокоуровневые конструкции, такие как `lock`, `Monitor`, `Mutex` и др., если только вы не уверены в необходимости ручного управления барьерами для достижения максимальной производительности.

Для lock-free программирования понимание барьеров памяти необходимо, но в таких случаях также рекомендуется использовать готовые concurrent коллекции и примитивы из `System.Collections.Concurrent` и `System.Threading`, которые уже правильно реализованы.

Промежуточные итоги

- Interlocked = Атомарность + Полный барьер памяти
- Барьер Interlocked гарантирует:
 - Видимость операций до Interlocked другим потокам
 - Завершение операций до барьера до начала операций после
 - Атомарность самой операции
- Interlocked не заменяет все барьеры - для несвязанных операций могут потребоваться дополнительные барьеры

Вопросы для проверки

1. Что такое **memory barrier**?
2. Какие типы барьеров бывают?
3. Как барьеры памяти связаны с Interlocked?



Вопросы



если есть вопросы



если вопросов нет

Ключевое слово `volatile` и класс `Volatile`

Ключевое слово **volatile**

Ключевое слово **volatile** применяется к полям класса и гарантирует, что:

- Чтение поля всегда возвращает самое актуальное значение из памяти
- Запись в поле немедленно становится видимой другим процессорам
- Предотвращает переупорядочивание операций с volatile-полем

Синтаксис:

```
private volatile bool _isRunning;  
private volatile int _counter;
```



Ключевое слово `volatile` и класс `Volatile`

Ключевое слово **`volatile`** применяется к полям класса и гарантирует, что:

- Чтение поля всегда возвращает самое актуальное значение из памяти
- Запись в поле немедленно становится видимой другим процессорам
- Предотвращает переупорядочивание операций с `volatile`-полем

Синтаксис:

```
private volatile bool _isRunning;  
private volatile int _counter;
```

Класс `Volatile` - статический класс в пространстве имен `System.Threading`, предоставляющий методы для явного управления барьерами памяти.



Ключевое слово **volatile** и класс **Volatile**

- **volatile** гарантирует, что компилятор не будет кэшировать значение и что операции чтения и записи не будут переупорядочены. Однако **volatile** не обеспечивает атомарность для неатомарных операций (например, инкремент).
- Класс **Volatile** предоставляет методы `Read` и `Write`, которые работают аналогично, но с более четкой семантикой.



Ключевые различия

Аспект	volatile поле	Класс Volatile
Применение	Только к полям класса	К любым переменным через ref
Типы	Ограниченный набор	Любые типы (generic)
Гибкость	Все операции с полем имеют барьеры	Явный контроль над барьерами
Производительность	Барьеры на каждую операцию	Барьеры только когда явно указано
Читаемость	Более чистый синтаксис	Более явный контроль



LIVE

Ключевые моменты

- `volatile` поля обеспечивают барьеры памяти для всех операций чтения/записи
- `Volatile` класс позволяет явно контролировать где нужны барьеры
- Double-checked locking требует `volatile` или `Volatile` для корректной работы
- CAS операции часто сочетаются с `Volatile.Read` для оптимизации
- Кэширование - основной сценарий использования `volatile/Volatile`



Вопросы для проверки

1. В чем особенность ключевого слова `volatile`?
2. Как используется класс `Volatile`?
3. Как ключевое слово `volatile` и класс `Volatile` связаны с барьерами памяти?



Вопросы



если есть вопросы



если вопросов нет

Сравнение производительности lock-free и lock-based подходов

LIVE

Результаты сравнения

Производительность при разной конкуренции:

- Низкая конкуренция (1-2 потока): lock-based $\approx$ lock-free
- Средняя конкуренция (4-8 потоков): lock-free быстрее на 20-50%
- Высокая конкуренция (16+ потоков): lock-free быстрее в 2-5 раз

Затраты памяти:

- Lock-based: overhead объекта блокировки (~24-48 bytes)
- Lock-free: минимальный overhead (только для atomic операций)

Стоимость операций:

- Lock acquisition: 20-50 ns
- Interlocked operation: 5-20 ns
- CAS loop: 10-100 ns (зависит от конкуренции)

Масштабируемость:

- Lock-based: линейно ухудшается с ростом конкуренции
- Lock-free: лучше масштабируется при высокой конкуренции



Практические выводы

- Для простых операций используйте Interlocked методы
- При высокой конкуренции предпочитайте lock-free подходы
- Для сложной логики используйте lock-based подходы
- Всегда измеряйте производительность в реальных сценариях
- Балансируйте между производительностью и сложностью кода

Сравнение показывает что не существует "серебряной пули" - выбор между lock-free и lock-based подходами зависит от конкретного сценария использования и требований приложения.



**Подведём
итоги занятия**

Цели вебинара

К концу занятия вы сможете

1. Научиться разрабатывать потокобезопасные структуры данных без использования блокировок;
2. Применять класс Interlocked для атомарных операций над примитивными типами;
3. Понимать роль Volatile и барьеров памяти для обеспечения корректности многопоточного кода.



Список материалов для изучения

[Жизнь без блокировок: не только lock-free и не только коллекции](#)

[Введение в lock-free программирование](#)

[Fear and Loathing in Lock-Free Programming](#)

[Проблема ABA в lock-free алгоритмах](#)

[Как создавать lock-free структуры данных в C# на базе CAS и Thread.Volatile](#)

[Lockfree, Waitfree, Obstruction-free, Atomic-free Algorithms](#)

[Неблокирующие межпроцессные коммуникации](#)

[Lock-free algorithms: The opportunistic cache](#)

[Lock-free коллекции в .NET 6](#)

[Lockless Programming Considerations for Xbox 360 and Microsoft Windows](#)

[Разработка синхронизированных многопоточных приложений на C# и .NET](#)

[Проблема ABA в lock-free алгоритмах](#)

[Атомарные операции с Interlocked](#)

[Атомарные операции, безопасность потоков и состояние гонки в C#](#)

[Модель памяти C# в теории и на практике](#)

[Interlocked vs Lock in C#](#)





Отправьте в чат эмодзи, который отражает ваше настроение после занятия



Продолжите высказывание: теперь я умею/знаю...



Что планируете использовать в работе?

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

OTUS

